Задание 22 ЕГЭ по информатике — подробный разбор алгоритма решения

> Это расшифровка «шпаргалки» из переписки: 8 шагов решения задания 22 через электронную таблицу (`.ods`), с объяснением **зачем** нужен каждый шаг и **как именно** работают формулы, в первую очередь `ВПР` (VLOOKUP).

---

0. Итог в двух словах

- В файле `22.ods` — 25 процессов, у каждого есть длительность и список процессов, от которых он зависит.
- Процесс можно запускать только тогда, когда **все** его зависимости уже завершились; каждый процесс стартует в **самый ранний допустимый момент**.
- Нужно найти, сколько процессов выполняется **одновременно на 7-й миллисекунде**.
- **Ответ: 8.**

Общая идея решения — три действия:

1. **Развернуть** зависимости каждого процесса (строка вида `8;6;15`) в отдельные столбцы.
2. Для каждой зависимости **найти время её окончания** — это делает `ВПР` .
3. Из времён окончания зависимостей вычислить **своё** начало и конец, «раскрасить» миллисекунды единичками и посчитать их в нужном столбце.

---

1. Условие задачи (что дано)

В файле хранится таблица процессов. Три столбца:

Столбец	Смысл
ID процесса B	идентификатор процесса
Время выполнения процесса B (мс)	сколько миллисекунд он длится
ID процесса(-ов) A	от кого он зависит; несколько ID пишутся через `;` , а `0` означает «не зависит ни от кого»

Правило зависимости: **если процессу B нужны результаты процесса A, то A и B выполняются строго последовательно** — B не может начаться раньше, чем A закончится. Процессы, не связанные зависимостями, могут идти параллельно.

Считать, что каждый процесс начинается **в самое раннее допустимое время**. Нумерация миллисекунд — **с 1**.

Типовой пример из условия (он только иллюстрирует формат данных, для ответа не используется):

ID	Время (мс)	Зависит от
1	3	0
2	4	1
3	2	2; 4
4	5	0
5	8	1; 4
6	3	1

Здесь сказано: «процесс 3 начинается на 8-й мс, заканчивается на 9-й мс». Проверим: процесс 4 длится 5 мс и стартует с 1-й мс → занимает мс 1–5; процесс 1 занимает мс 1–3; процесс 2 зависит от 1 → стартует с 4-й мс, длится 4 мс → занимает мс 4–7. Процесс 3 зависит от 2 и 4: самый поздний из них (процесс 2) освобождается после 7-й мс, значит процесс 3 стартует на **8-й** мс и при длительности 2 занимает мс 8–9. Всё сходится.

---

2. Модель времени — самое важное для понимания формул

Введём для каждого процесса две величины:

- `start` — номер первой миллисекунды, в которую он работает;
- `end` — номер последней миллисекунды, в которую он работает.

Правила:

1. **Свободный процесс** (зависимостей нет) стартует с 1-й мс: `start = 1`. Именно поэтому в таблице появляется «виртуальная» нулевая миллисекунда — «до начала работы».
2. **Зависимый процесс** стартует сразу после того, как завершится **самая поздняя** из его зависимостей: `start = max(end зависимостей) + 1`.

`+1` — потому что если зависимость занимала 7-ю мс, то на 7-й мс процесс ещё занят, и новый может начаться только на 8-й.

3. **Конец** процесса: `end = start + длительность − 1`.
   Например, старт на 8-й мс при длительности 2 → занимает мс 8 и 9 → `end = 9`.

Отсюда удобная переформулировка, которая и используется в таблице:

```
end = max(конец зависимостей) + длительность
start = end − длительность + 1   =   max(конец зависимостей) + 1
```

> **Тонкость, которая путает почти всех.** В таблице есть столбец `I`, который считается как `J − B`. Это **не** номер первой рабочей миллисекунды, а «момент готовности» — последняя миллисекунда, которую занимали зависимости. Реально процесс работает в миллисекундах `I + 1 … J` (интервал `(I; J]`).
> У свободного процесса `I = 0`, и он работает с 1-й мс.

---

3. Данные из файла `22.ods`

Длительности и зависимости 25 процессов (столбцы A, B и C–E после разворота):

ID	Время, мс	Зависит от
1	4	3; 25
2	3	8; 6; 15
3	1	15; 22
4	4	7; 14
5	2	15; 8
6	1	8; 13; 16
7	6	17
8	1	25; 3; 22
9	3	17; 19; 2
10	1	19; 8
11	6	25
12	6	22
13	1	25; 16; 15
14	1	11; 19
15	3	17; 25
16	4	25

ID	Время, мс	Зависит от
17	2	— (0)
18	6	22
19	5	22
20	3	22; 10; 12
21	6	17
22	4	— (0)
23	4	18
24	4	21; 22; 25
25	5	— (0)

Максимум зависимостей — **три**, поэтому после разворота появится ровно три столбца с ID зависимостей.

---

4. Итоговая раскладка столбцов

Именно к такой структуре приводит алгоритм:

Столбец	Что в нём	Откуда берётся
A	ID процесса	исходные данные
B	длительность	исходные данные
C, D, E	ID 1-й, 2-й, 3-й зависимости	шаг 1 (разделение по `;`)
F, G, H	время **окончания** соответствующей зависимости	шаг 2 (`ВПР`)
I	момент готовности процесса (`J - B`)	шаг 4
J	время окончания процесса	шаг 3
K, L, M, …	по одному столбцу на каждую миллисекунду	шаги 5–7

---

5. Пошаговый разбор шпаргалки

Шаг 1. «Разделить по столбцам столбец с зависимостями»

Зачем. В одной ячейке лежит строка вроде `8;6;15`. Чтобы с каждой зависимостью можно было работать отдельно (искать её время окончания), её надо

разложить по отдельным ячейкам: `8`, `6`, `15`.

Как. Выделяем столбец C → меню **Данные** → **Текст по столбцам** (в LibreOffice Calc: *Данные* → *Текст по столбцам*), разделитель — точка с запятой `;`. Результат записывается в C, D, E (первое значение — в исходный столбец).

Результат. Теперь у каждого процесса до трёх «ячеек-зависимостей». Если зависимостей меньше, лишние ячейки пустые; если зависимостей нет — стоит `0`.

Шаг 2. `ВПР` — превращаем ID зависимости во время

Зачем. Нам нужно знать, **когда заканчивается** каждая зависимость. В таблице время окончания процесса хранится в столбце `J` (см. шаг 3), а связь между процессами задана через ID. Значит, по ID зависимости нужно «достать» значение из столбца `J` той строки, где в столбце `A` стоит этот ID. Ровно это и делает `ВПР`.

Формула (как в шпаргалке):

```
=ЕСЛИОШИБКА(ВПР(C1;$A$2:$I$26;10);0)
```

Полный корректный вариант, который мы разбираем (см. замечание ниже):

```
=ЕСЛИОШИБКА(ВПР(C2;$A$2:$J$26;10;0);0)
```

Что делает `ВПР` (VLOOKUP) — синтаксис:

```
ВПР( искомое_значение ; таблица ; номер_столбца ; [интервал_просмотра] )
```

Разберём по аргументам:

1. `C2` **— искомое значение.** Это ID зависимости, которую мы сейчас «расшифровываем». `ВПР` пойдёт искать это число в **первом столбце** указанной таблицы, то есть в столбце `A`.
2. `$A$2:$J$26` **— таблица поиска.** Весь блок данных: строки 2–26, столбцы A–J. Знаки `$` делают ссылку **абсолютной**: при копировании формулы вниз и вправо границы таблицы не «уползут». Это критично — таблица поиска одна и та же для всех ячеек.
3. `10` **— номер столбца внутри таблицы, значение из которого надо вернуть.** Столбцы нумеруются **внутри диапазона**, начиная с первого: `A=1, B=2, C=3, D=4, E=5, F=6, G=7, H=8, I=9, J=10`. Значит `10` — это столбец `J`, то есть **время окончания** процесса.

Логика чтения: «найди строку, где ID совпал, и верни мне время окончания этого процесса».

4. `0` **(или** `ЛОЖЬ`**) — точное совпадение.** Если аргумент опущен, `ВПР` ищет «приблизительно» и требует отсортированных данных. Для поиска по ID надёжнее указывать `0`. В шпаргалке четвёртый аргумент опущен — здесь это работает, потому что ID идут по возрастанию, но лучше писать `0` явно.

Зачем снаружи `ЕСЛИОШИБКА`**.** Если в ячейке зависимости стоит `0` (зависимостей нет) или ячейка пустая, то `ВПР` не найдёт такое значение и вернёт ошибку `#Н/Д`. Обёртка `ЕСЛИОШИБКА( … ; 0)` заменяет любую ошибку на `0`. А `0` — это ровно то, что нужно: «никакая зависимость не задерживает, можно начинать с 1-й мс».

Почему формула вводится «диапазоном из стольких же столбцов». После разворота
зависимости лежат в `C`, `D`, `E` — три столбца. Значит нужно три столбца с результатом: `F`, `G`, `H`. Мы выделяем `F2:H26`, пишем формулу один раз и заполняем, а `ВПР` в каждой ячейке ссылается на **столбец слева**:

- `F2 = ВПР(C2; …)` — время окончания 1-й зависимости;
- `G2 = ВПР(D2; …)` — 2-й;
- `H2 = ВПР(E2; …)` — 3-й.

Относительная ссылка `C2` при протягивании вправо автоматически превращается в `D2`, `E2`, а при протягивании вниз — в `C3`, `C4`, … Именно поэтому формулу можно ввести сразу на весь блок.

> **Важное замечание про диапазон.** В шпаргалке записано `$A$2:$I$26`, но номер столбца `10` указывает на `J`. Если диапазон действительно заканчивается на `I` (9 столбцов), то номер `10` выходит за его границы, `ВПР` вернёт ошибку, а `ЕСЛИОШИБКА` превратит её в `0` — и все результаты будут нулями. Чтобы формула возвращала время окончания, диапазон обязан включать столбец `J`, то есть быть `$A$2:$J$26`. Похоже, в шпаргалке просто опечатка.

Результат шага. В `F`, `G`, `H` для каждого процесса лежат времена окончания его зависимостей (или `0`, если зависимости нет).

Шаг 3. Время окончания процесса: `=МАКС(F2:G2)+B2`

Формула (как в шпаргалке):

```
=МАКС(F2:G2)+B2
```

Полный вариант с учётом трёх столбцов зависимостей:

```
=МАКС(F2:H2)+B2
```

Зачем и как работает.

- `F2:H2` — времена окончания **всех** зависимостей процесса.
- `МАКС(...)` берёт **самую позднюю** из них. Это ключевая идея: процесс не может начаться, пока не закончится **последняя** из его зависимостей. Если взять не максимум, а, например, первую зависимость, то процесс «убежит вперёд» и начнётся
  раньше, чем ему разрешено.
- `+B2` прибавляет **собственную длительность**. В итоге получаем номер последней миллисекунды, которую занимает процесс: он стартует сразу после `МАКС(...)` и работает ещё `B2` миллисекунд.

Проверим на процессе 3 (строка 4): зависимости 15 и 22. Процесс 15 заканчивается на 8-й мс, процесс 22 — на 4-й. `МАКС(8; 4) = 8`, длительность процесса 3 равна 1, итого `8 + 1 = 9` — процесс заканчивается на 9-й мс. Совпадает со скриншотом.

> **Почему это не «циклическая ссылка», хотя выглядит как цикл.** `J` зависит от `F:H`, а `F:H` через `ВПР` — от столбца `J`. Цикла здесь нет, потому что каждый процесс ссылается не на себя, а на **другие** процессы (свои зависимости). Граф зависимостей ацикличен, поэтому таблица спокойно вычисляет всё по цепочке. При
> этом `ВПР` ищет зависимость по ID, а не по номеру строки, так что порядок строк в файле значения не имеет.
>
> В шпаргалке записан диапазон `F2:G2` — это вариант для двух столбцов с зависимостями. Если разворот дал три столбца (`C:E`), максимум надо брать по `F:H`, иначе третья зависимость не будет учтена.

Результат шага. Столбец `J` — время окончания каждого процесса.

Шаг 4. «Пропущенный» столбец: `=J2-B2`

Формула:

```
=J2-B2
```

Зачем. Из времени окончания нужно получить момент, **когда процесс становится готов к запуску**. Так как `end = max(зависимости) + длительность`, то

```
J2 − B2 = max(конец зависимостей)
```

— это последняя миллисекунда, которую занимали зависимости. Процесс же реально работает начиная со **следующей** миллисекунды: `(I; J]`.

На шаге 3 мы специально «отступили» от края на один пустой столбец, чтобы потом заполнить его этой формулой. В итоге:

- `I` — момент готовности (последняя мс зависимостей);
- `J` — момент окончания самого процесса.

Например, для процесса 1: `J = 13`, `B = 4`, значит `I = 9`. Процесс 1 работает в миллисекундах 10–13.

Шаги 5–6. Много узких столбцов и нумерация миллисекунд

Зачем. Нужно построить «сетку времени»: один столбец = одна миллисекунда. Тогда в каждой клетке можно отметить, работает ли там конкретный процесс, а затем сложить отметки по столбцу и узнать число параллельных процессов.

Как. Правее столбца `J` создаём несколько десятков узких столбцов (начинаем с `K`) и в первой строке проставляем нумерацию `1, 2, 3, …` — это номера миллисекунд. Заголовок `K1 = 1`, `L1 = 2`, `M1 = 3`, … Значит **7-й столбец этой сетки — это миллисекунда 7**.

Шаг 7. Формула «процесс работает в эту миллисекунду»

Формула:

```
=ЕСЛИ(И($I2<K$1;K$1<=$J2);1;"")
```

Как читать. Условие `И(A; B)` истинно, только если верны **оба** условия:

1. `$I2 < K$1` — номер миллисекунды **строго больше** момента готовности;
2. `K$1 <= $J2` — номер миллисекунды **не больше** времени окончания.

Вместе это означает: миллисекунда попадает в интервал `(I; J]` — ровно те миллисекунды, в которые процесс работает. Если да — ставим `1`, иначе оставляем пустую строку `""` (пустая строка, а не `0`, чтобы столбцы легко читались и суммировались).

Почему ссылки такие «хитрые»:

- `$I2` и `$J2` — столбец `I` / `J` зафиксирован знаком `$`, а строка относительна. Поэтому при протягивании формулы **вниз** каждая строка берёт свои `I` и `J`.

- `K$1` — строка `1` зафиксирована, а столбец относителен. Поэтому при протягивании **вправо** `K$1` превращается в `L$1`, `M$1`, … — то есть формула сама подставляет номер очередной миллисекунды из заголовка столбца.

Именно это сочетание `$` и позволяет написать формулу один раз и растянуть её на весь блок `K2:…26`.

Шаг 8. Считаем 7-й столбец

Зачем. Теперь в каждом столбце сетки стоят единицы у работающих процессов. Число параллельных процессов на данной миллисекунде = **количество единиц в столбце** (в LibreOffice Calc его показывает строка состояния, либо можно посчитать `СУММ` / `СЧЁТ`).

Выделяем **седьмой** столбец сетки (миллисекунда 7) и считаем единицы глазами. Их **8** — это и есть ответ.

---

6. Проверка и ответ

Полный расчёт по модели из раздела 2 даёт такую картину:

Миллисекунда	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Работает процессов	3	3	4	4	6	8	**8**	8	7	7

На 7-й миллисекунде одновременно работают процессы
7, 11, 12, 15, 16, 18, 19, 21 — их восемь.

Ответ: 8

---

7. Частые ошибки

1. **Забыть про нумерацию с 1.** Свободный процесс стартует на 1-й мс, а не на 0-й. Отсюда «виртуальный» ноль в столбце `I` и строгое неравенство `I < t`.
2. **Взять не максимум по зависимостям, а любую из них.** Процесс обязан ждать самую позднюю зависимость.
3. **Не расширить `МАКС` на все столбцы зависимостей.** Если разворот дал три столбца, диапазон должен быть `F:H`, а не `F:G`.

4. **Неверный номер столбца в `ВПР`.** Номер считается **внутри диапазона** и должен указывать на столбец со временем окончания (`J → 10`).
5. **Забыть `ЕСЛИОШИБКА`.** Без неё `0` и пустые зависимости дадут `#Н/Д` и «сломают» всю цепочку вычислений.
6. **Неточное совпадение в `ВПР`.** Для поиска по ID лучше явно указывать четвёртый аргумент `0` .
7. **Путать «момент готовности» `I` и первый рабочий момент.** Процесс работает в `(I; J]` , то есть начиная с `I + 1` .

---

8. Как проверить ответ другим способом

Ту же самую модель легко запрограммировать (полезно для самопроверки). Логика: для каждого процесса `start = max(конец зависимостей) + 1` , `end = start + длительность − 1` ,
затем подсчитать, сколько интервалов `[start; end]` накрывают каждую миллисекунду.
Такой расчёт для данных из `22.ods` даёт ту же таблицу из раздела 6 и ответ **8**.

Резюме одним абзацем

Разворот зависимостей по столбцам даёт отдельные ID; `ВПР` по каждому ID находит время окончания соответствующей зависимости; `МАКС` из этих времён плюс собственная длительность даёт время окончания процесса (`J`), а `J − B` — момент готовности (`I`). Сетка узких столбцов с номерами миллисекунд и формула `=ЕСЛИ(И($I2<K$1;K$1<=$J2);1;"")` отмечают, в какие миллисекунды процесс работает, и подсчёт единиц в столбце миллисекунды 7 даёт ответ **8**.